

Real-Time Embedded Systems (#41505)

Laboratory Module: Review of C, Modular Programming in C, Makefiles and Unit Testing.

Paulo Pedreiras, pbrp@ua.pt
Pedro Fonseca, pf@ua.pt

Department of Electronics, Telecommunications and Informatics
University of Aveiro
Academic year 2026/2027

Laboratory Module: Review of C, Modular Programming in C and Makefiles. Unit tests.

Total work duration: 10 hours

Use of AI: Constrained AI model. AI can be used to clarify specific issues, complementing the bibliography and class materials. AI should not be used to solve the exercises of fix bugs in the code produced by students.

Notes: additional exercises can be fond in the recommended bibliography

Learning Outcomes:

- Review and apply core C programming features
- Correct use of variable scope (static, extern, volatile)
- Manipulate pointers and dynamic memory
- Implement and manipulate linked lists
- Develop modular programs with header and source files
- Write and use Makefiles for project compilation

Theme 1: Variable scope and interfaces

Exercise 1.1: Keeping local

Implement a C function that generates a random number (between 0 and 100) and keeps track of how many times it has been called. Global variables are not allowed.

In the main call the function a few times and check if it works correctly.

Use the prototype:

void MyRand(int *value, int *ncalls)

where “value” is the random number and ncalls is the number of times the function was called (including the present one)

Exercise 1.2: Sharing variables across modules

Create a program with a main file and a module (“configData.c” and “configData.h”) that holds configuration data (for the case just a single variable, maxConnections, uint16_t).

For the moment, the configData module just contains the declaration of the maxConnections variable and provides a function to initialize it to zero, which should be called before using the variable.

Create a main program that initializes maxConnections (using the function above), and reads and modifies **directly** its value. Add a few lines fo initializing and assigning values, to verify if data manipulation is done correctly.

Exercise 1.3: Specifying interfaces/encapsulation

The approach used in the previous exercise allows direct variable manipulation among modules, which is only feasible in simple software, as it requires specific knowledge about module internals and does not allow to validate variable contents.

Modify the previous software by including in configData a set of interface functions:

void configInit(void); // Init maxConnections to zero

void configSetMaxConnections(uint16_t value);// Set the value of maxConnections

uint16_t configGetMaxConnections(void); // Get the value of maxConnections

Of course, maxConnections should be “hidden” inside the configData module. The main program shall not have direct access to it.

Again, add a few lines fo initializing and assigning values, to verify if data manipulation is done correctly.

The use of Unity, for unit testing, is recommended.

Theme 2: Pointers and Memory Management

Exercise 2.1: dynamic array statistics

Write a function that:

I. Allocates memory for an array of N integers comprised between 1 and M,

II. Initialize it with random values (comprised between 1 and M), and

III. Returns the pointer to the array, and the average, max, and min of its elements.

M and N are integer arguments, passed to the function.

Write a main function that allows to check the correct operation of the function, e.g. by printing the array elements and the return values for inspection.

Exercise 2.2: Swap with pointer-to-pointer

Implement a function to swap two integer values using pointers. Write a main function that allows to check it correct operation.

Exercise 2.3: String reversal with pointer arithmetic

Reverse a string in place using pointer arithmetic.

Theme 3: Linked lists and encapsulation

Exercise 3.1: Basic Linked List

Create a program to manage a singly linked list with insert, delete, and print (separate) functions.

Each node should store an integer value.

Write a main function to check if the linked list functions work correctly.

Exercise 3.2: Linked List in Modular Files

Refactor the linked list into separate .c/.h files using encapsulation.

Use Unity to verify the correct implementation of the module.

Theme 4: Final Project – Word Analyzer

Task: Word Analyzer

Build a modular program to analyze a text file for lines, words, characters, and word frequency.

The program should return a summary like:

10 lines

300 words

931 characters

Word frequency:

- and: 31

- student: 14

…

Interpreting the report, we can say that the text file has 10 lines, 300 words, 931 characters, the word “and” occurs 31 times, the word “student” occurs 14 times, ...

If the time is not enough, you can skip the computation of the word frequency.

Structure: Break into main.c, file_utils.c/.h, word_list.c/.h, and Makefile.

Additional work: Sort words by frequency and add unit tests.